

به نام خدا

معماری کامپیوتر

تمرین کامپیوتری شماره 2

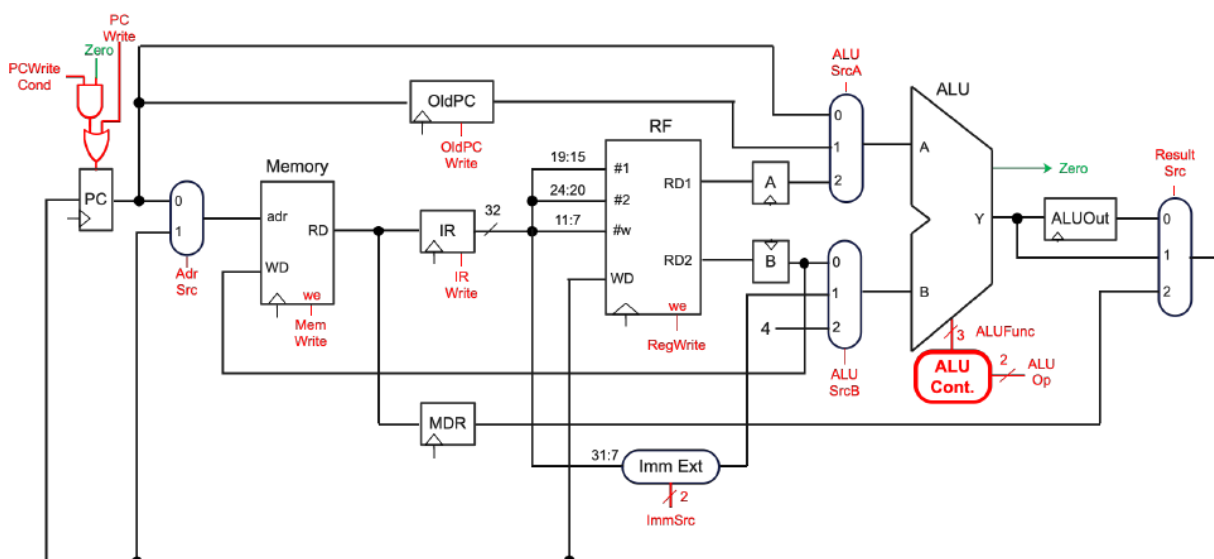
سید امین اسماعیلی زکی 810103356

علی دهقانی دهقی 810103414

طراحی مسیر داده:

1. برای دستورات زیر می‌توانیم از مسیر داده تدریس شده استفاده کنیم (که مسیر داده آن تمام آنها را پشتیبانی می‌کند):

- R-Type: add, sub, and, or, slt
- I-Type: lw, addi, ori, slti
- S-Type: sw
- B-Type: beq



- دستورات R-Type مستقیماً از طریق ALU قابل پیاده‌سازی هستند.
- همچنین برای دستورات I-Type به جز lw سه مورد دیگر از طریق Imm Ext و توابع ALU قابل اجرا هستند.
- دستورات beq, sw, lw هم از دستورات پایه هستند که از قبل در مسیر داده ایجاد شدند.

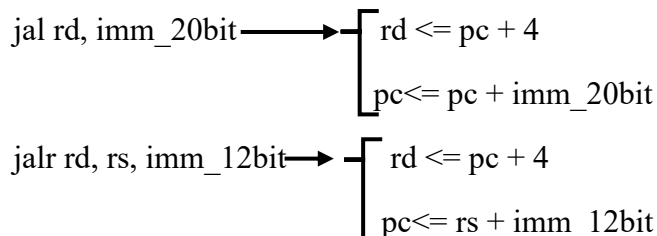
2. حالا برای اضافه کردن دستورات جدید داریم:

- I-Type: xori

- برای دستور بالا مجبور هستیم که یک تابع جدید به ALU اضافه کنیم چون در غیر این صورت برای پیاده سازی آن نیازمند چندین کلاک استفاده از ALU و ذخیره مقادیر این دستور در رجیستر ها هستیم (که انجام این کار با RF کنونی بسیار دشوار است). پس تابع xor را به ALU اضافه می کنیم.

- J-Type: jal, I-Type: jalr

- تفاوت این دو دستور تنها در نحوه مقداردهی pc است:



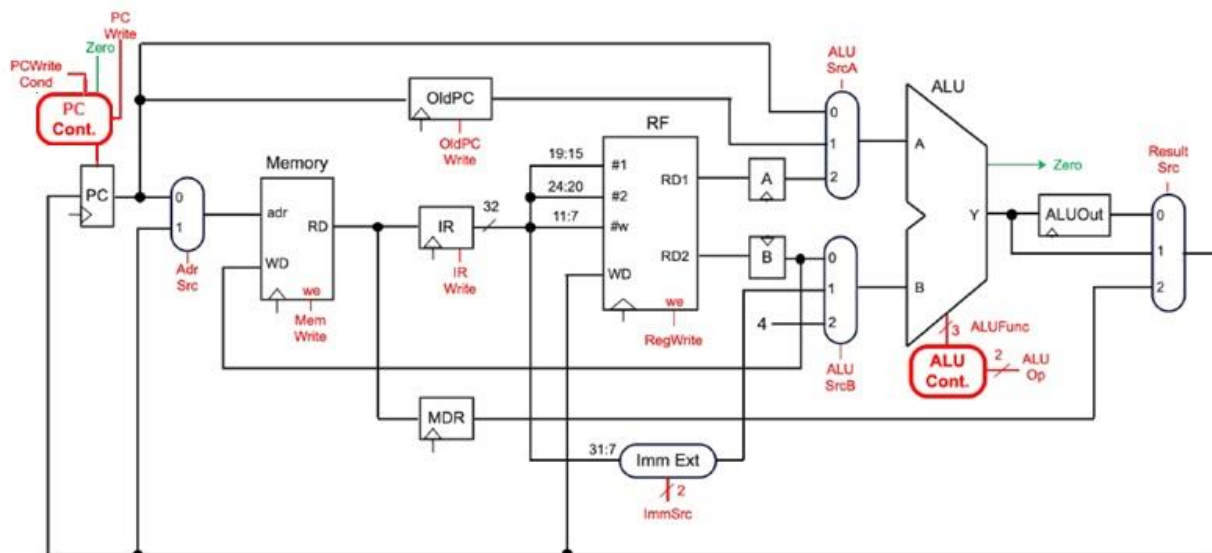
- و برای هر دو مسیر اجرای آنها از قبل در مسیر داده صفحه قبل موجود است (زیرا برای مقدار دهی pc در سیکل اول استفاده می شود) ولی امکان استفاده از آن برای این دو دستور نیز وجود دارد، به این صورت:
- برای jal کافی است ورودی اول ALU را OldPC قرار دهیم و ورودی دوم را عدد ثابت 4 و خروجی آنها را پس از ذخیره در رجیستر ALUout در RF و آدرس رجیستر rd ذخیره کنیم و در سیکل بعدی ورودی اول ALU را همان OldPC و ورودی دوم را خروجی Imm Ext قرار دهیم و خروجی آن که در ALUout ذخیره می شود را در pc لود کنیم.
- برای دستور jalr برای مقداردهی رجیستر rd همان حالتی که برای دستور jal اجرا کردیم اجرا میکنیم ولی برای مقداردهی pc ورودی اول ALU را به جای OldPC ، rs را انتخاب میکنیم (که همان A است) و برای بقیه موارد دقیقاً مثل دستور قبل.

- B-Type: bne, blt, bge

- با توجه به اینکه از قبل دستور beq را داریم پیاده سازی آنها خیلی شبیه به آن و ساده خواهد بود:
- bne: کافی است برعکس شرط beq را بررسی کنیم یعنی در صورتی پرش را انجام دهیم که اختلاف دو عدد صفر نباشد و در نتیجه سیگنال zero برابر صفر باشد.
- blt: برای آن از تابع ALU slt استفاده میکنیم به این صورت اگر ورودی A کوچکتر از B باشد خروجی ALU یعنی Y برابر 1 می شود و در نتیجه سیگنال zero صفر می شود. پس میتوانیم براساس zero تصمیم گیری کنیم که اگر صفر بود پرش انجام شود.
- bge: برعکس دستور قبل عمل میکنیم چون در این دستور اگر ورودی A بزرگتر یا مساوی B باشد باید پرش را انجام دهیم و اگر این اتفاق بیفتد خروجی Y در حالت slt برابر 0 می شود که باز هم میتوانیم از سیگنال zero استفاده کنیم که اگر یک باشد پرش را انجام دهیم.
- با توجه به اضافه شدن این دستورات برای سیگنال نوشتن PC تصمیم گیری به سادگی قبل نیست و نمی توان با استفاده از همان دو گیت کار را انجام داد پس یک بخش کنترلر مجزا برای آن در نظر میگیریم به نام PC Cont.

3. کشیدن مسیر داده:

- طبق توضیحات بخش قبل هیچ تغییری در مسیر داده نیاز نیست و در نتیجه مسیر داده نهایی تصویر صفحه قبل است. (لازم به ذکر است که بخش کنترلر ALU به عنوان ورودی func3 و func7 را هم دریافت می کند و همچنین بخش کنترلر PC هم به عنوان ورودی func3 را دریافت میکند ولی در تصویر نمایش داده نشده اند)



طراحی کنترلر:

1. ALUFunc:

ALU Func	Y
000	$A+B$ (add)
001	$A-B$ (sub)
010	$A \& B$ (and)
011	$A B$ (or)
100	$A \leq B$ (set less than)
101	$A \oplus B$ (xor)

- طبق توضیحاتی که قبل تر داده شد یک تابع جدید XOR به حالت های قبلی اضافه می کنیم:

2. ImmSrc:

- با اضافه کردن دستورات از نوع پرش jal باید حالت جدیدی برای استخراج مقدار immediate اضافه کنیم که جدول آن به صورت زیر می شود:

Imm Src	Imm Ext	Inst Type
00	$\{20\{Inst[31]\}\}, Inst[31:20]$	I-Type (lw)
01	$\{20\{Inst[31]\}\}, Inst[31:25], Inst[11:7]$	S-Type (sw)
10	$\{19\{Inst[31]\}\}, Inst[31], Inst[7], Inst[30:25], Inst[11:8], 1'60$	B-Type (bex)
11	$\{12\{Inst[31]\}\}, Inst[19:12], Inst[20], Inst[30:21], 1'60$	J-Type (jal)

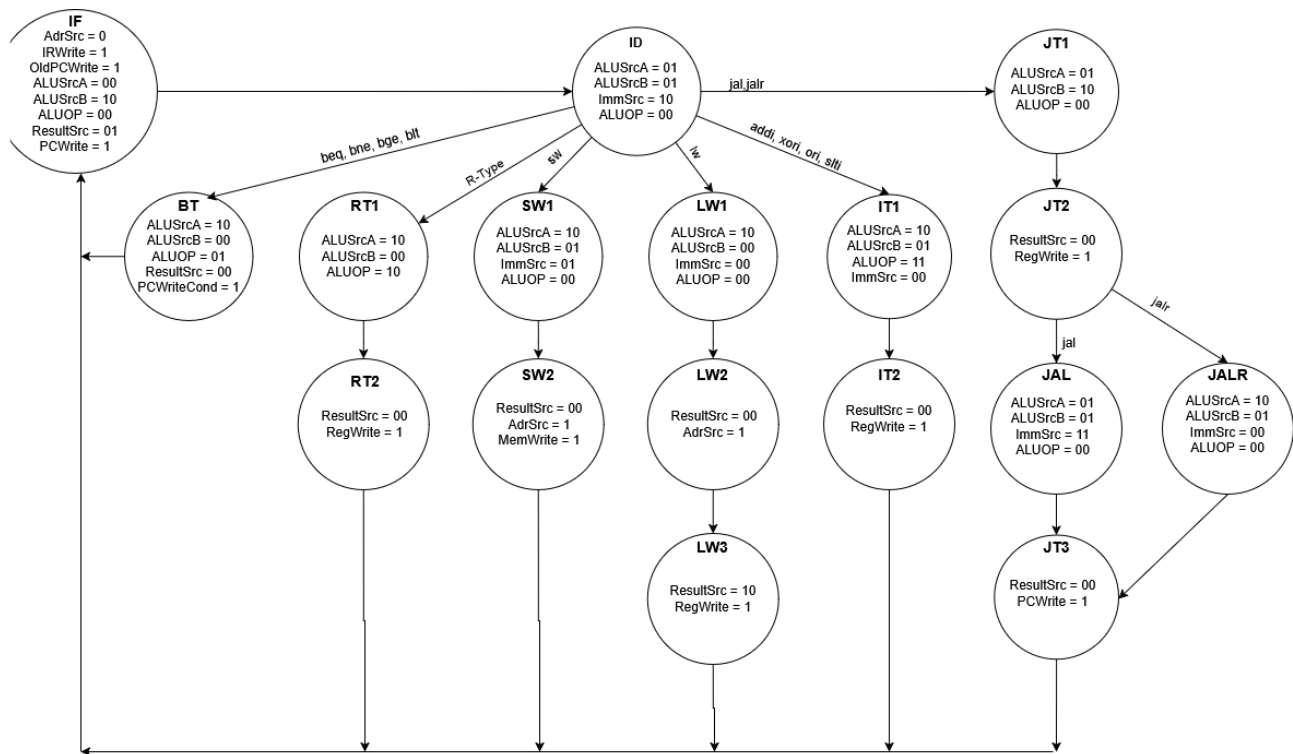
3. وضعیت ALUOP (و ALUFunc) و ImmSrc برای هر دستور بر اساس opcode, func3, func7.

	op	op code	func3	func7	Imm Src	ALUop	ALUFunc
R_Type	add	0110011	000	0000000	XX	10	000
	sub		000	0100000	XX	10	001
	and		111	0000000	XX	10	010
	or		110	0000000	XX	10	011
	slt		010	0000000	XX	10	100
I_Type	lw	0000011	010	-	00	00	000
	addi	0010011	000		00	11	000
	xori		100		00	11	101
	ori		110		00	11	011
	slti		010		00	11	100
	jalc	1100111	000		00	XX	XXX
S_Type	sw	0100011	010	-	01	00	000
J_Type	jal	1101111	-	-	11	XX	XXX
B_Type	beq	1100011	000	-	10	01	001
	bne		001		10	01	001
	blt		100		10	01	100
	bge		101		10	01	100

4. شکل کلی کنترلر:

○ کنترلر از سه بخش تشکیل شده است که شامل:

- Main controller: بخش اصلی کنترلر است که تمام سیگنال های لازم را به جز ALUFunc ایجاد میکند.
- PCController: قسمت مجزا که تنها برای کنترل سیگنال لود رجیستر pc استفاده می شود و از دو سیگنال خروجی کنترلر اصلی استفاده میکند: PCWrite, PCWriteCond و همچنین خروجی zero, ALU که با استفاده از گیت های پایه and و or تصمیم گیری را انجام میدهد. همچنین func3 را هم به عنوان ورودی دریافت می کند زیرا برای تصمیم گیری برای پرش های شرطی براساس نوع آن (که از روی func3 مشخص میشود) باید تصمیم بگیرد که از سیگنال zero به چه شکل استفاده کند (برای مثال برای beq اگر zero صفر بود باید پرش را انجام دهد ولی برای bne اگر صفر نبود).
- ALUController: با دریافت سیگنال ALUOP و func3 و func7 سیگنال ALUFunc را ایجاد میکند.



پیاده سازی حافظه یا همان Memory:

1. در این پروژه بر خلاف قبلی، حافظه ما فقط در یک فایل ذخیره می شود و این فایل باید توانایی آن را داشته باشد که هم حافظه داده ها و هم حافظه دستورات پردازنده (PC) باشد.
2. برای این کار یکی از این دو باید در ابتدا و دیگری باید در خانه ی n ام حافظه آغاز به کار کند. ما در آزمایش قبلی کد سورت کردن پردازنده را طراحی کرده بودیم که در آن حافظه ها از مقدار 0 آغاز می شد. برای آن که بتوانیم مجدد از آن کد صفر و یکی استفاده کنیم، حافظه داده را از همان اول فایل آغاز کرده و حافظه دستورات PC را به جلوتر منتقل می کنیم و از خانه 300 آن ها را قرار می دهیم. برای این کار باید درون کد PC پیاده سازی کنیم:

```
module PC
(
    input clk, reset, enable,
    input [31:0] PC_in,
    output reg [31:0] PC_out
);

always @(posedge clk)
begin
    if (reset)
        PC_out <= 32'd300;

    else if (enable)
        PC_out <= PC_in;
end
endmodule
```

3. با توجه به کد، مشخص است که هنگام شروع حافظه به جای خانه اول، دستورات را از خانه 300 بر می دارد که این اجازه را به ما خواهد داد که هر دو حافظه درون یک فایل باشند و هم بتوانیم از کد سورت تمرین قبل در آن استفاده کنیم. ماژول حافظه نیز به صورت زیر خواهد بود و از بار قبلی تغییری در آن به نسبت به DataMem ایجاد نشده است:

```
module Memory
(
    input clk,
    input [31:0] Adr,
    input [31:0] write_data,
    input we,
    output [31:0] read_data
);

reg [7:0] mem [0:1023];

always @(posedge clk)
begin
    if (we)
        {mem[Adr + 3], mem[Adr + 2], mem[Adr + 1], mem[Adr]} <= write_data;
end
```

```
assign read_data = {mem[Adr + 3], mem[Adr + 2], mem[Adr + 1], mem[Adr]};

initial
begin
    $readmemb("memory.mem", mem);
end

integer file;
integer i;

final begin
    file = $fopen("memory.mem", "w");

    for(i = 0; i < 1024; i = i + 1)
    begin
        $fwrite(file, "%08b\n", mem[i]);
    end

    $fclose(file);
end

endmodule
```

پیاده سازی مسیر داده:

1. برای پیاده سازی مسیر داده، ابتدا تمام ماژول هایی که در آن آمده است را طراحی می کنیم. حافظه با این که در این بخش قرار ندارند و خارج پردازنده به آن نصب می شوند، ولی باز هم برای متصل کردن اتصالات درست آن ها را در این بخش طراحی می کنیم و فقط متصل کردنشان را به آخر کار موکول می کنیم:

- ALU
- Multiplexer2to1
- Multiplexer3to1
- PC → تفاوت آن با باقی رجیستر ها داشتن ریست برای آغاز سیمولیت است
- Register
- RegisterWithEnable
- RF
- Memory → در این جا طراحی می کنیم ولی در بیرون پردازنده آن را نصب می کنیم
- Imm_Ext

2. حال نوبت پیاده سازی مسیر داده است. کفایت ماژول های طراحی شده را (به جز حافظه را که در انتها با آن ها سر و کار داریم) به یک دیگر به صورت ساختاری متصل کنیم. اتصالات به کنترل و واحد های حافظه را به همراه کلاک نیز در این جا به صورت ورودی و خروجی در نظر خواهیم گرفت:

```
module DataPath
(
    input clk, reset, PCWrite, AddrSrc, RegWrite, OldPCWrite, IRWrite,
    input [1:0] ImmSrc, ALUSrcA, ALUSrcB, ResultSrc,
    input [2:0] ALUFunc,
    input [31:0] MemData,
    output zero,
    output [31:0] MemAddress, MemWriteData, inst
);

    wire [31:0] Result, PC_out, RegData1, RegData2, ALU_A, ALU_B, ALU_out,
    Imm_out, OldPC_out, ALUOut_out, A_out, B_out, MDR_out;

    assign MemWriteData = B_out;

    PC stage0 (clk, reset, PCWrite, Result, PC_out);
    RF stage1 (clk, inst[19:15], inst[24:20], inst[11:7], Result, RegWrite,
    RegData1, RegData2);

    ALU stage2 (ALU_A, ALU_B, ALUFunc, ALU_out, zero);
    Imm_Ext stage3 (inst[31:7], ImmSrc, Imm_out);

    RegisterWithEnable IR (clk, IRWrite, MemData, inst);
    RegisterWithEnable OldPC (clk, OldPCWrite, PC_out, OldPC_out);

    Register A (clk, RegData1, A_out);
```



```

Register B (clk, RegData2, B_out);
Register MDR (clk, MemData, MDR_out);
Register ALUOut (clk, ALU_out, ALUOut_out);

Multiplexer2to1 stage4 (PC_out, Result, AddrSrc, MemAddress);

Multiplexer3to1 stage5 (PC_out, OldPC_out, A_out, ALUSrcA, ALU_A);
Multiplexer3to1 stage6 (B_out, Imm_out, 32'd4, ALUSrcB, ALU_B);
Multiplexer3to1 stage7 (ALUOut_out, ALU_out, MDR_out, ResultSrc, Result);

endmodule

```

پیاده سازی کنترلر:

قدم اول، پیاده سازی MainController:

1. این پروژه، دیگر کنترل اصلی ما به صورت قبل نخواهد بود. در بار گذشته صرفاً طراحی به صورت تک سیکل بود و همین که ما با کمک operation code دستور را متجه می شدیم، می توانستیم خروجی های مورد نیاز را قرار دهیم و دستور ها تک سیکل اجرا و تمام می شدند.
2. اما در این پروژه، دستور ها به صورت مولتی سایکل پیاده می شوند، یعنی ما باید استیت تعریف کنیم، هر بار بر اساس دستور به استیت مورد نظر برویم، بعضی مواقع دستورات را شناسایی کنیم تا بتوانیم استیت مورد نظر را پیدا کنیم.
3. طبق طراحی قرار گرفته در قسمت طراحی واحد کنترل، ما نیاز به 17 استیت داریم. پس برای شناسایی آن ها به یک عدد 5 بیتی نیاز مندیم. در طراحی از اسم هایی روان استفاده می کنیم تا همه قدم ها مشخص باشد. در هر استیت مشخص می کنیم که قدم بعدی آن چیست. اگر می توانست به چند استیت برود، استیت هدف را به کمک operation code شناسایی می کنیم و به استیت درست می رویم. از طراحی هافمن برای راحتی و فهم بیشتر استفاده می کنیم. برای این پیاده سازی نیاز به دو always داریم، درون اولی باید در ابتدا استیت را مقدار دهی اولیه کنیم و به استیت بعدی برویم که با کلاک کار می کند و در دومی باید دستورات را پیاده سازی و استیت بعدی را مشخص کنیم. در نهایت کد این بخش به صورت زیر در می آید:

```

module MainController
(
    input clk, reset,
    input [6:0] operation_code,
    output reg PCWrite, AddrSrc, RegWrite, OldPCWrite, IRWrite, PCWriteCond,
    MemWrite,
    output reg [1:0] ImmSrc, ALU_operation, ALUSrcA, ALUSrcB, ResultSrc
);

    parameter IF = 5'b00000, ID = 5'b00001, BT = 5'b00010, RT1 = 5'b00011, RT2 =
    5'b00100, SW1 = 5'b00101, SW2 = 5'b00110, LW1 = 5'b00111;
    parameter LW2 = 5'b01000, LW3 = 5'b01001, IT1 = 5'b01010, IT2 = 5'b01011, JT1
    = 5'b01100, JT2 = 5'b01101, JAL = 5'b01110, JALR = 5'b01111;
    parameter JT3 = 5'b10000;

```

```

reg [4:0] stage, next_stage;

always @(posedge clk) begin
    if (reset)
        stage <= IF;
    else
        stage <= next_stage;
end

always @(*)
begin
    PCWrite = 0; AdrSrc = 0; RegWrite = 0; OldPCWrite = 0; IRWrite = 0;
    PCWriteCond = 0; MemWrite = 0;
    ALUSrcA = 2'b00; ALUSrcB = 2'b00; ImmSrc = 2'b00; ALU_operation = 2'b00;
    ResultSrc = 2'b00;
    next_stage = IF;

    case (stage)
        IF:
        begin
            PCWrite = 1; AdrSrc = 0; IRWrite = 1; OldPCWrite = 1;
            ALUSrcA = 2'b00; ALUSrcB = 2'b10; ALU_operation = 2'b00;
            ResultSrc = 2'b01;
            next_stage = ID;
        end
        ID:
        begin
            ALUSrcA = 2'b01; ALUSrcB = 2'b01; ALU_operation = 2'b00; ImmSrc =
2'b10;

            case(operation_code)
                7'b0110011: next_stage = RT1;
                7'b0000011: next_stage = LW1;
                7'b0010011: next_stage = IT1;
                7'b1100111: next_stage = JT1;
                7'b0100011: next_stage = SW1;
                7'b1101111: next_stage = JT1;
                7'b1100011: next_stage = BT;
            endcase
        end
        BT:
        begin
            PCWriteCond = 1;
            ALUSrcA = 2'b10; ALUSrcB = 2'b00; ALU_operation = 2'b01;
            ResultSrc = 2'b00;
            next_stage = IF;
        end
        RT1:

```

```

begin
    ALUSrcA = 2'b10; ALUSrcB = 2'b00; ALU_operation = 2'b10;
    next_stage = RT2;
end
RT2:
begin
    RegWrite = 1;
    ResultSrc = 2'b00;
    next_stage = IF;
end
SW1:
begin
    ALUSrcA = 2'b10; ALUSrcB = 2'b01; ALU_operation = 2'b00; ImmSrc =
2'b01;
    next_stage = SW2;
end
SW2:
begin
    AdrSrc = 1; MemWrite = 1;
    ResultSrc = 2'b00;
    next_stage = IF;
end
LW1:
begin
    ALUSrcA = 2'b10; ALUSrcB = 2'b00; ALU_operation = 2'b00; ImmSrc =
2'b00;
    next_stage = LW2;
end
LW2:
begin
    AdrSrc = 1;
    ResultSrc = 2'b00;
    next_stage = LW3;
end
LW3:
begin
    RegWrite = 1;
    ResultSrc = 2'b10;
    next_stage = IF;
end
IT1:
begin
    ALUSrcA = 2'b10; ALUSrcB = 2'b01; ALU_operation = 2'b11; ImmSrc =
2'b00;
    next_stage = IT2;
end
IT2:
begin

```

```

        RegWrite = 1;
        ResultSrc = 2'b00;
        next_stage = IF;
    end
    JT1:
    begin
        ALUSrcA = 2'b01; ALUSrcB = 2'b10; ALU_operation = 2'b00;
        next_stage = JT2;
    end
    JT2:
    begin
        RegWrite = 1;
        ResultSrc = 2'b00;
        case(operation_code)
            7'b1100111: next_stage = JALR;
            7'b1101111: next_stage = JAL;
        endcase
    end
    JAL:
    begin
        ALUSrcA = 2'b01; ALUSrcB = 2'b01; ALU_operation = 2'b00; ImmSrc =
2'b11;
        next_stage = JT3;
    end
    JALR:
    begin
        ALUSrcA = 2'b10; ALUSrcB = 2'b01; ALU_operation = 2'b00; ImmSrc =
2'b00;
        next_stage = JT3;
    end
    JT3:
    begin
        PCWrite = 1;
        ResultSrc = 2'b00;
        next_stage = IF;
    end
endcase
end
endmodule

```

قدم دوم، پیاده سازی ماژول Controller:

1. ما برای پیاده سازی کنترل نیاز است سه ماژول را به صورت جدا گانه پیاده سازی کنیم و با اتصال این سه ماژول به یک دیگر، کنترلر ما به صورت ترکیبی ساخته خواهد شد:

- اولین ماژولی که ما نیاز داریم، MainController است. این ماژول موظف است بر اساس operation code تمام خروجی های کنترلری مورد نیاز را به جز دو خروجی PCWrite و ALU_function فراهم آورد. برای فراهم آوردن این دو نیاز به اطلاعاتی فراتر از operation code است ولی برای تشخیص این دو دستور، باز هم operation code نیاز است. پس این ماژول سه خروجی با نام های PCWriteCond، PC_write و ALU_operation برای محاسبه دو خروجی مورد نیاز فراهم می کند.
- دومین ماژول ALUController است. این ماژول موظف است با گرفتن ALU_operation و func3 و func7، نوع دستور را به صورت کامل شناسایی کند و خروجی مناسب را برای ALU بفرستد تا محاسبات به صورت درست انجام شود.
- سومین ماژول PCController است. این ماژول موظف است با گرفتن PC_write، PCWriteCond، func3 و همچنین سیگنال ورودی zero، کنترل کننده ورودی PC را کنترل کند که آیا روی رجیستر PC چیزی نوشته شود یا خیر. در اصل سیگنال خروجی PCWrite را به صورت درست مقدار دهی می کند.
- ماژول MainController ما این بار دیگر مانند قبل نیست و ورودی کلاک را می گیرد و بر اساس استیت ها، عمل های مناسب آن ها را که در دیاگرام بالا آمده است اجرا می کند.

2. بعد از طراحی ریز ماژول های کنترلری بالا، نوبت به آن می رسد که این سه ماژول را به هم وصل کنیم و یک کنترلر ترکیبی کامل داشته باشیم:

```
module Controller
(
    input clk, reset, zero,
    input [31:0] inst,
    output PCWrite, AddrSrc, RegWrite, OldPCWrite, IRWrite, MemWrite,
    output [1:0] ImmSrc, ALUSrcA, ALUSrcB, ResultSrc,
    output [2:0] ALU_function
);

    wire PCWriteCond, PC_write;
    wire [1:0] ALU_operation;

    MainController stage0 (clk, reset, inst[6:0], PC_write, AddrSrc, RegWrite,
OldPCWrite, IRWrite, PCWriteCond, MemWrite, ImmSrc, ALU_operation, ALUSrcA,
ALUSrcB, ResultSrc);
    ALUController stage1 (inst[14:12], inst[31:25], ALU_operation, ALU_function);
    PCController stage2(inst[14:12], PCWriteCond, PC_write, zero, PCWrite);

endmodule
```

پیاده سازی پردازنده RISC-V:

- حال وقت آن می شود که متصل کردن کنترلر و مسیر داده، پردازنده RISC-V را کامل کنیم:

```
module RISCv
(
    input clk, reset,
    input [31:0] MemData,
    output [31:0] MemAddress, MemWriteData,
    output MemWrite
);
    wire PCWrite, AdrSrc, RegWrite, OldPCWrite, IRWrite;
    wire [1:0] ImmSrc, ALUSrcA, ALUSrcB, ResultSrc;
    wire [2:0] ALUFunc;
    wire zero;
    wire [31:0] inst;

    DataPath stage0 (clk, reset, PCWrite, AdrSrc, RegWrite, OldPCWrite, IRWrite,
ImmSrc, ALUSrcA, ALUSrcB, ResultSrc, ALUFunc, MemData, zero, MemAddress,
MemWriteData, inst);
    Controller stage1 (clk, reset, zero, inst, PCWrite, AdrSrc, RegWrite,
OldPCWrite, IRWrite, MemWrite, ImmSrc, ALUSrcA, ALUSrcB, ResultSrc, ALUFunc);
endmodule
```

پیاده سازی TopModule:

- بعد از کامل کردن پردازنده، نوبت به آن رسیده که آن را به واحد حافظه ی Memory را به پردازنده متصل کرده و Top module، خود را پیاده سازی کنیم که تمامی امکانات را داشته باشد. کد این بخش هم به صورت زیر خواهد بود:

```
module TopModule
(
    input clk, reset
);
    wire [31:0] MemData, MemAddress, MemWriteData;
    wire MemWrite;

    RISCv stage0 (clk, reset, MemData, MemAddress, MemWriteData, MemWrite);
    Memory stage1 (clk, MemAddress, MemWriteData, MemWrite, MemData);
endmodule
```

- برای اجرای این ماژول فقط کافیست یک تست بنچ بنویسیم که clk را جابجا کند و ریست را در اولین سیکل کلاک 1 قرار دهد و باقی مواقع صفر. دیگر باقی کار ها را خودش به صورت کامل انجام خواهد داد.

2. پیاده سازی کد C++:

```
arr[10] // Assuming the array is given like this

for (int i = 0; i < n - 1; i++) {
    int minIndex = i;
    for (int j = i + 1; j < n; j++) {
        if (arr[j] < arr[minIndex]) {
            minIndex = j;
        }
    }
    swap(arr[i], arr[minIndex]);
}
```

3. تبدیل به دستورات مجاز RISC_V:

○ با استفاده از جدول جزییات سیگنال های کنترلر که func7, func3, op دستورات در آن ذکر شده است دستورات را به فرم زیر تبدیل میکنیم:

1.	addi x8, x0, 10	# x8 = n = 10
2.	addi x9, x0, 9	# x9 = n - 1 = 9
3.	add x5, x0, x0	# x5 = i = 0
4.	add x25, x0, x0	# x25 = i offset = 0
5.	outer_loop: bge x5, x9, done	# if i >= 9, exit loop
6.	add x11, x0, x5	# minIndex = i
7.	add x26, x0, x25	# minIndex_offset = i_offset
8.	addi x6, x5, 1	# j = i + 1
9.	addi x27, x25, 4	# j_offset = i_offset + 4
10.	inner_loop: bge x6, x8, end_inner	# if j >= 10, exit inner loop
11.	lw x28, 0(x27)	# x28 = arr[j]
12.	lw x29, 0(x26)	# x29 = arr[minIndex]
13.	bge x28, x29, skip	# if arr[j] >= arr[minIndex], skip update
14.	add x11, x0, x6	# minIndex = j
15.	add x26, x0, x27	# minIndex_offset = j_offset
16.	skip: addi x6, x6, 1	# j++
17.	addi x27, x27, 4	# j_offset += 4
18.	jal x0, inner_loop	# jump to inner_loop
19.	end_inner: lw x28, 0(x25)	# x28 = arr[i]
20.	lw x29, 0(x26)	# x29 = arr[minIndex]
21.	sw x29, 0(x25)	# arr[i] = old arr[minIndex]
22.	sw x28, 0(x26)	# arr[minIndex] = old arr[i]
23.	addi x5, x5, 1	# i++
24.	addi x25, x25, 4	# i_offset += 4
25.	jal x0, outer_loop	# jump to outer_loop
26.	done:	

- برای مقادیر پرش با دستورات bge و jal باید دستورات را به صورت 4 حطی فرض کنیم (چون در ادامه هر دستور به 4 خط 8 بیتی تبدیل می شود) و بر اساس آن آدرس خطی که به آن باید پرش کند را پیدا کنیم که به صورت زیر می شود:

- done = +84
- end_inner = +36
- skip = +12
- inner_loop = -32
- outer_loop = -80

○ نتیجه نهایی:

```
1.      addi x8, x0, 10           # x8 = n = 10
2.      addi x9, x0, 9           # x9 = n - 1 = 9
3.      add x5, x0, x0           # x5 = i = 0
4.      add x25, x0, x0          # x25 = i_offset = 0
5.  outer_loop:  bge x5, x9, 84    # if i >= 9, exit loop
6.      add x11, x0, x5          # minIndex = i
7.      add x26, x0, x25         # minIndex_offset = i_offset
8.      addi x6, x5, 1           # j = i + 1
9.      addi x27, x25, 4         # j_offset = i_offset + 4
10. inner_loop:  bge x6, x8, 36    # if j >= 10, exit inner loop
11.      lw x28, 0(x27)          # x28 = arr[j]
12.      lw x29, 0(x26)          # x29 = arr[minIndex]
13.      bge x28, x29, 12        # if arr[j] >= arr[minIndex], skip update
14.      add x11, x0, x6         # minIndex = j
15.      add x26, x0, x27        # minIndex_offset = j_offset
16. skip:      addi x6, x6, 1     # j++
17.      addi x27, x27, 4        # j_offset += 4
18.      jal x0, -32             # jump to inner_loop
19. end_inner:  lw x28, 0(x25)    # x28 = arr[i]
20.      lw x29, 0(x26)          # x29 = arr[minIndex]
21.      sw x29, 0(x25)          # arr[i] = old arr[minIndex]
22.      sw x28, 0(x26)          # arr[minIndex] = old arr[i]
23.      addi x5, x5, 1          # i++
24.      addi x25, x25, 4        # i_offset += 4
25.      jal x0, -80             # jump to outer_loop
26. done:
```

4. تبدیل به دستورات باینری براساس op code های RISC_V:

```
1. 0000000010100000000010000010011
2. 0000000010010000000010010010011
3. 0000000000000000000000001010110011
4. 000000000000000000000000110010110011
5. 00000100100100101101101001100011
6. 00000000010100000000010110110011
7. 00000001100100000000110100110011
8. 00000000000100101000001100010011
9. 00000000010011001000110110010011
10. 00000010100000110101001001100011
11. 00000000000011011010111000000011
12. 00000000000011010010111010000011
13. 00000001110111100101011001100011
14. 00000000011000000000010110110011
15. 00000001101100000000110100110011
16. 00000000000100110000001100010011
17. 00000000010011011000110110010011
18. 11111110000111111111000001101111
19. 00000000000011001010111000000011
20. 00000000000011010010111010000011
21. 00000001110111001010000000100011
22. 00000001110011010010000000100011
23. 00000000000100101000001010010011
24. 00000000010011001000110010010011
25. 11111011000111111111000001101111
```

5. تبدیل به خطوط 8 بیتی:

- در این تبدیل به این صورت عمل میکنیم که بیت های پرارزش تر دستور در خطوط پایین تر آن دستور قرار بگیر که بتوان با استفاده از خط زیر آنها را استخراج کرد:

`assign RD = {mem[Adr + 3], mem[Adr + 2], mem[Adr + 1], mem[Adr]}`

- در نتیجه داریم:

```
301. 00010011
302. 00000100
303. 10100000
304. 00000000
305. 10010011
306. 00000100
307. 10010000
308. 00000000
309. 10110011
310. 00000010
311. 00000000
312. 00000000
313. 10110011
314. 00001100
315. 00000000
316. 00000000
317. 01100011
318. 11011010
319. 10010010
320. 00000100
321. 10110011
322. 00000101
323. 01010000
324. 00000000
325. 00110011
326. 00001101
327. 10010000
328. 00000001
329. 00010011
330. 10000011
331. 00010010
332. 00000000
333. 10010011
334. 10001101
```

```
335. 01001100
336. 00000000
337. 01100011
338. 01010010
339. 10000011
340. 00000010
341. 00000011
342. 10101110
343. 00001101
344. 00000000
345. 10000011
346. 00101110
347. 00001101
348. 00000000
349. 01100011
350. 01010110
351. 11011110
352. 00000001
353. 10110011
354. 00000101
355. 01100000
356. 00000000
357. 00110011
358. 00001101
359. 10110000
360. 00000001
361. 00010011
362. 00000011
363. 00010011
364. 00000000
365. 10010011
366. 10001101
367. 01001101
368. 00000000
```

```
369. 01101111
370. 11110000
371. 00011111
372. 11111110
373. 00000011
374. 10101110
375. 00001100
376. 00000000
377. 10000011
378. 00101110
379. 00001101
380. 00000000
381. 00100011
382. 10100000
383. 11011100
384. 00000001
385. 00100011
386. 00100000
387. 11001101
388. 00000001
389. 10010011
390. 10000010
391. 00010010
392. 00000000
393. 10010011
394. 10001100
395. 01001100
396. 00000000
397. 01101111
398. 11110000
399. 00011111
400. 11111011
```

6. نوشتن آرایه 10 عنصری از اعداد صحیح:

○ برای آرایه از اعداد زیر استفاده میکنیم:

۷۷, ۱- , ۲۳, ۵۰- , ۸, ۵- , ۱۰۰ , ۰ , ۱۲- , ۴۵ ○

○ که فرم مکمل دو آنها به صورت زیر است:

[illegible]

○ مانند قسمت دستورات الگوریتم مرتب سازی در اینجا هم هر عدد را به صورت 32 بیتی در 4 خط می نویسیم که هر

خط 8 و خطوط پایین پر ارزش تر هستند:

```

1. 110011110
2. 11111111
3. 11111111
4. 11111111
5. 11110100
6. 11111111
7. 11111111
8. 11111111
9. 00000000
10. 00000000
11. 00000000
12. 00000000
13. 01100100
14. 00000000
15. 00000000
16. 00000000
17. 11111011

```

```
18. 11111111
19. 11111111
20. 11111111
21. 00001000
22. 00000000
23. 00000000
24. 00000000
25. 00101101
26. 00000000
27. 00000000
28. 00000000
29. 00010111
30. 00000000
31. 00000000
32. 00000000
33. 11111111
34. 11111111
```

```
35.11111111
36.11111111
37.01001101
38.00000000
39.00000000
40.00000000
```